CS 4782: Deep Learning – Project Summary Report
Group Members: Connie Chen (cc2524), Aiden Lee (akl72), Jason Zhou (jxz3), Srikanth Karaikal (sk2783)

GitHub Repository Link: https://github.com/cc2524-2109/lora

1. Introduction

In the past, fine-tuning large language models (LLMs) had been extremely expensive. This is because these models have unimaginably large weight matrices and large numbers of trainable parameters so updating all of these trainable parameters was memory-intensive and slow. While there are existing parameter-efficient methodologies (adapter layers, prefix tuning, etc.), these methods incur inference latency or reduce the available input length.

To address this issue, researchers of the paper "*LoRA: Low-Rank Adaptation of Large Language Models (Hu et al., 2021)*" introduced LoRA, an incredibly impactful Parameter-Efficient Fine-Tuning (PEFT) that that freezes all pre-trained weights and injects trainable low-rank decomposition matrices $A \in \mathbb{R}^{rxd}$ and $B \in \mathbb{R}^{dxr}$ in parallel with each target weight matrix. This paper suggests that LoRA allows LLMs to train with, up to 10,000x, fewer parameters while achieving better or comparable results than prefix-training and adapter layers without the aforementioned drawbacks of existing parameter-efficient methodologies.

2. Chosen Result

We aim to reproduce *Table 3 — GPT-2 Medium and Large on E2E NLG Challenge (NLG)* of the paper, which compares the performance and number of trainable parameters of GPT-2 Medium and Large models adapted with different fine-tuning methods on the E2E NLG Challenge. In choosing to do this, we strive to validate one of the core findings of the paper: that a rank-4 adapter on Wq and Wv alone achieves results comparable to that of full fine-tuning and other fine-tuning methods (Full fine-tuning (FT), Adapter tuning variants (AdapterL, AdapterH), etc.) using 1000× fewer trainable parameters.

3. Methodology

Given our goal to re-implement the results of *Table 3*, our methodology closely aligns with that of the paper. We start by creating a GPT2 wrapper class that injects LoRA into the transformer weights of the GPT2 model. Specifically, the wrapper class decomposes the pre-trained weight matrix into the input projection matrices (Q, K, and V matrices) and applies LoRA to both Q and V. As in the paper, we didn't apply LoRA to K because it does not have as significant of an effect as opposed to Q and V.

The actual LoRA implementation consists of two low-rank decomposition matrices ($A \in \mathbb{R}^{rxd}$ and $B \in \mathbb{R}^{dxr}$) that, when multiplied, result in a weight update matrix ($\Delta W \in \mathbb{R}^{dxd}$, where $\Delta W = BA$), with the same dimensions as Q or V. At initialization, Matrix B is set to be all 0s. Matrix A consists of random numbers drawn from a Gaussian distribution. During training, instead of training on the pretrained weight matrix from the GPT2 model, we freeze these weights and only train A and B. Afterward, we multiply A and B to get the total weight update ΔW, which is then added to the original Q and V matrices. It is important to note that both Q and V each get their own "copy" of LoRA, so there are two sets of A and B matrices. Thus, we have two distinct ΔWs: one to add to Q, and one to add to V. Lastly, the updated Q and V are then combined with the original K to give us the updated transformer weights.

Finally, we train the LoRA-injected GPT2 on the E2E NLG dataset, a benchmark for end-to-end data-to-text generation that trains systems to produce natural language descriptions from structured inputs in the restaurant domain. The models are then evaluated on 5 different metrics (BLEU, NIST, METEOR, ROUGE-L, and CIDEr), used to measure the quality of generated text by comparing it to human-written reference text. BLEU and NIST focus on n-gram overlap, with NIST giving more weight to informative/rarer words. METEOR evaluates semantic similarity using stemming and synonym matching. ROUGE-L measures overlap based on the longest common subsequence. CIDEr emphasizes consensus across multiple references.

Note: Tables & Visualizations are listed at the end for organizational purposes.

4. Results & Analysis

Our results confirm that the paper's core claim still holds. Despite the metric gap, our results reproduce the qualitative finding that adapting only Wq and Wv with r = 4 yields just 0.35M trainable parameters for GPT-2 Medium — roughly 1000× fewer than full fine-tuning — while targeting comparable NLG quality. It shows that LoRA, despite a significantly lower number of parameters, can still learn meaningful representations and achieve competitive performance. However, our implementation still consistently underperforms compared to the paper's implementation of LoRA. This is likely due to differences in training setup, such as a larger number of training runs and randomization of training data during optimization, which can improve generalization and lead to better validation performance.

This paper is a monumental paper regarding Parameter-Efficient Fine-Tuning (PEFT) on LLMs, with over 31,000 citations Google Scholar alone. Our results largely validates the aforementioned core claim yet, it was still unable to fully replicate the paper's results, mirroring a well-documented challenge in the ML reproducibility literature: papers report results under optimal conditions that are difficult to reconstruct. The LoRA paper is already more transparent than most, with Table 11 listing explicitly hyperparameters, yet the gap persisted.

5. Reflections

During this project, we under-estimated the required resources and time needed to replicate the results due to ambiguities in the paper. As a result, in our attempt to achieve results that matches that of the paper, we experiment extensively with each of the following:

- E2E Input Format / Prompt Template: The paper never shows how slot-value pairs are serialized into a string before tokenization. As such, we experimented with different prefixes ("input/output") and delimiters (",", "|", etc.)
- A Matrix Initialization: The paper says only "random Gaussian initialization for A" (Section 4.1) but it gives no standard deviation, no mention of Kaiming uniform, etc. As a result, we tested the model on different A matrix initialization (kaiming, torch.rand, etc.)
- Tokenizer Settings: There is also no mention of max input sequence length for training, or how inputs longer than the context window are handled (truncation vs. skipping).
- Metric Implementation: Neither the specific library nor version is cited for the metrics. METEOR in particular has several variants (with/without stemming, paraphrase tables). We initially used a different version compared to the paper, resulting in vastly different scores. Because we hadn't suspected this to be the issue, this resulted in a long period of confusion and debugging.

With the length of training time, the frequent time-out/disconnect of Google Colab, and the limited space on Google Drive (stops saving results once a high-percentage of storage is used), each iteration required a lot of resources and time. If we had more time/resources, we would have explored the following:

- Random Seeds: Section 5.4 says results are the "mean over 3 random seeds" but never states which seeds were used. With the proper resources, we would expand our implementation beyond simply one seed and allow multiple runs per seed.
- Validation Strategy During Training: Section 5.4 says "result for each run is taken from the best epoch" but doesn't specify which metric is used to select the best epoch or whether validation is done every epoch or every N steps. With the proper resources, we test various different strategies for the best results.

The paper deliberately limits LoRA to attention weights for simplicity, A potential future direction would be MLP adaptation of LoRA, which could potentially improve performance further. Another potential extension would be adaptive rank selection. Table 6 of the paper shows that r =1 suffices for some tasks while others need r = 8 or higher. The paper offers no principled way to choose rank (r) in advance. As such, a potential direction for future work is to develop methods that automatically learn the required rank for each weight matrix.

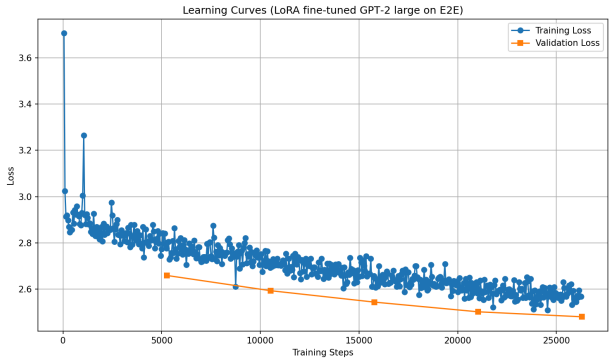
Note: Tables & Visualizations are listed at the end for organizational purposes.

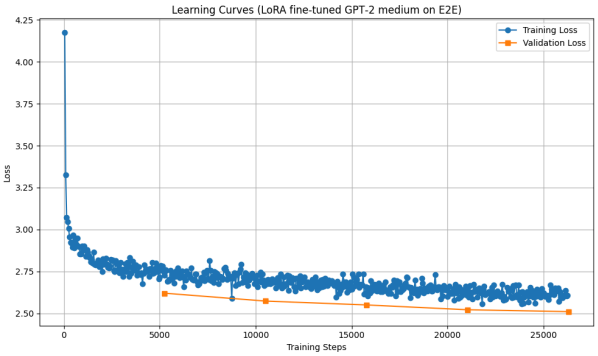
Tables & Visualizations

Paper Results + Our Results

Model & Method	Trainable Parameters	BLEU	NIST	METEOR	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (AdapterL)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (AdapterL)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (AdapterH)	11.09M	67.3±0.6	8.50±0.07	46.0±0.2	70.7±0.2	2.44±0.01
GPT-2 M (FTTop2)*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4±0.1	8.85±0.02	46.8±0.2	71.8±0.1	2.53±0.02
GPT-2 M (LoRA - Our Result)	0.35M	67.38	7.36	45.67	68.81	1.76
GPT-2 L (FT)*	774.03M	68.5	8.78	46	69.9	2.45
GPT-2 L (AdapterL)	0.88M	69.1±0.1	8.68±0.03	46.3±0.0	71.4±0.2	2.49±0.0
GPT-2 L (AdapterL)	23.00M	68.9±0.3	8.70±0.04	46.1±0.1	71.3±0.2	2.45±0.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4±0.1	8.89±0.02	46.8±0.2	72.0±0.2	2.47±0.02
GPT-2 L (LoRA - Our Result)	0.77M	67.78	7.35	45.27	68.83	1.75

Our Implementation Learning Curves





Note: Tables & Visualizations are listed at the end for organizational purposes.

Table 11 - Hyperparameter for Implementation from the Paper

Dataset	E2E	WebNLG	DART
		Training	
Optimizer		AdamW	
Weight Decay	0.01	0.01	0.0
Dropout Prob	0.1	0.1	0.0
Batch Size		8	
# Epoch		5	
Warmup Steps		500	
Learning Rate Schedule		Linear	
Label Smooth	0.1	0.1	0.0
Learning Rate		0.0002	
Adaptation		$r_q = r_v = 4$	
LoRA α		32	
		Inference	
Beam Size		10	
Length Penalty	0.9	0.8	0.8
no repeat ngram size		4	

References:

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. arXiv. https://arxiv.org/abs/2106.09685

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (n.d.). *GLUE: General Language Understanding Evaluation benchmark*. https://gluebenchmark.com/

Novikova, J., Dušek, O., & Rieser, V. (2017). The E2E dataset: New challenges for end-to-end generation. In *Proceedings of the 18th Annual Meeting of the Special Interest Group on Discourse and Dialogue* (pp. 201–206). Association for Computational Linguistics. https://arxiv.org/abs/1706.09254

Note: Tables & Visualizations are listed at the end for organizational purposes.